Turvavöö hoiatussüsteem C-keeles

Oct 23, 2026 · @Karmo Kask

Moodul otsustab igas tsüklis (nt iga 10 ms järel), kas süttib turvavöö märgutuli ja kas kõlab helisignaal. Kui mõni signaal on vigane või sõnum vanem kui 200 ms, jääb märgutuli igal juhul põlema. Testid: 34/34 läbitud, 100% rea- ja harukatvus, MISRA-kontroll 0 leidu.

Ülesanne ja reeglid

Ülesanne: kirjutada C-keeles turvavöö hoiatussüsteem, mis järgib standardeid MISRA C, AUTOSAR ja ISO 26262. Sisenditeks on `ignition_on`, `belt_state`, `seat_state`, `speed_raw` ja `age_ms`.

Igale reeglile on antud tunnus, mis esineb ka koodi kommentaarides ja testides. Nii saab iga nõude koodist ja testidest üles leida (ISO 26262 jälgitavus).

Tunnus	Reegel
REQ-SBR-001	Hoiatus on vajalik ainult siis, kui süüde on sees, iste on hõivatud ja turvavöö ei ole kinnitatud.
REQ-SBR-002	Alati vähemalt visuaalne hoiatus. Kui kiirus on üle 10 km/h, lisandub heli.
REQ-SBR-003	Kui mõni signaal on vigane või sõnum vanem kui 200 ms, peab visuaalne hoiatus töötama.

Failid

Kood on jagatud kuueks failiks: loogika, liides ja seadistus on eraldi (AUTOSAR-i tava).

Fail	Mis seal on
`Std_Types.h`	AUTOSAR-stiilis põhitüübid: `uint8`, `boolean`, `Std_ReturnType`, `E_OK`, `NULL_PTR`
`Sbr_Cfg.h`	Seadistatavad väärtused: 10 km/h lävi, 200 ms ajalimiit, kiiruse piirid
`Sbr.h`	Mooduli liides: sisendite koodid, vigade bitid, tüübid ja funktsiooni `Sbr_Evaluate()` deklaratsioon

Fail	Mis seal on
`Sbr.c`	Hoiatusloogika: neli funktsiooni
`test_Sbr.c`	Ühiktestid: 29 nõuetepõhist testi, 4 API testi ja ~170 miljoni kombinatsiooni võrdlus
`Makefile`	Käsud `make test`, `make coverage`, `make misra`

Nimi **SBR** = *Seat Belt Reminder* (turvavöö meeldetuletus). Kõik nimed algavad `Sbr_` või `SBR_`, et need ei läheks teiste moodulite nimedega segi.

Sisendid ja väljundid

Sisendid tulevad sõiduki siinilt (nt CAN) toorväärtustena. Kehtivaks loetakse ainult kokkulepitud koodid, kõik muu on viga.

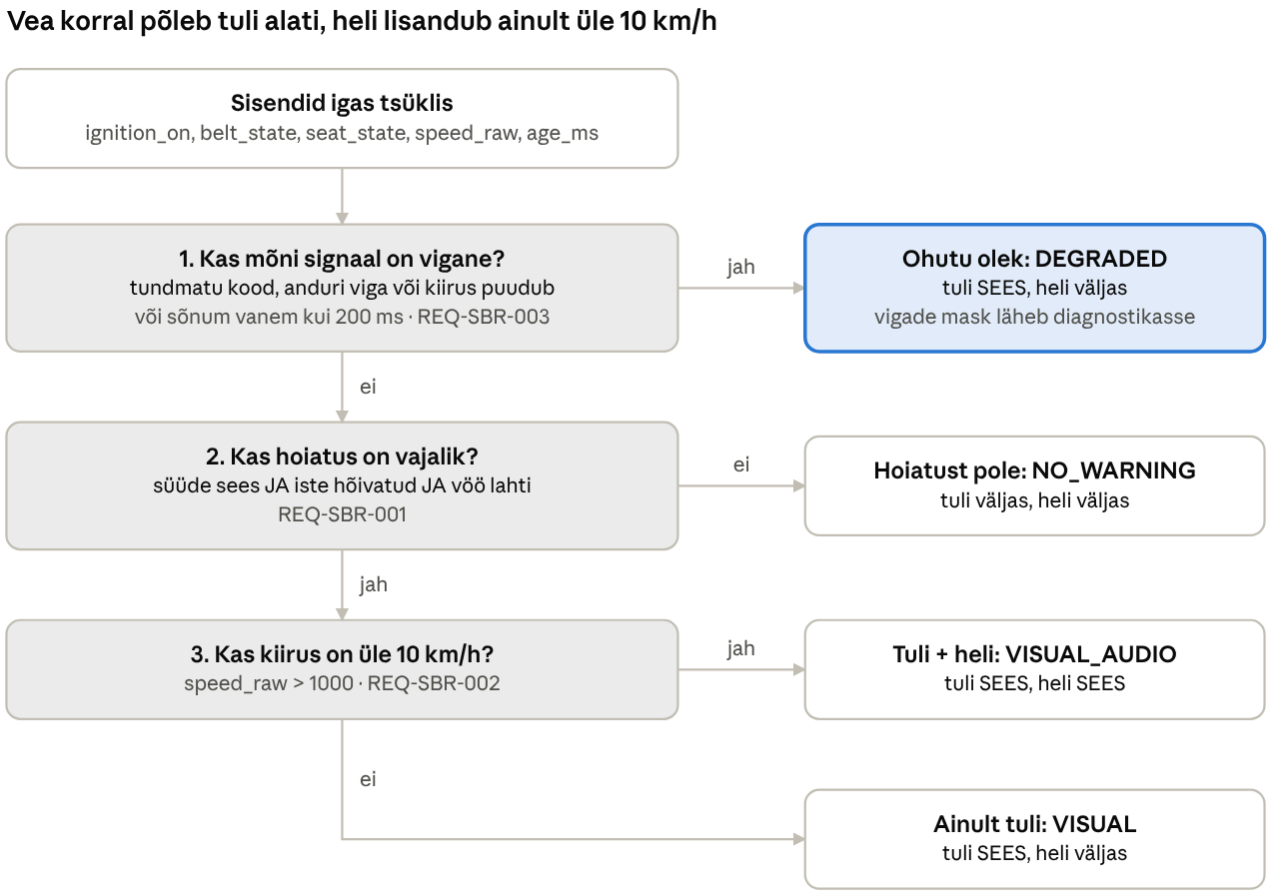
Sisend	Tüüp	Kehtivad väärtused	Loetakse veaks
`ignition_on`	`uint8`	0 = väljas, 1 = sees	iga muu väärtus (nt 0xFF = signaal puudub)
`belt_state`	`uint8`	0 = kinnitamata, 1 = kinnitatud	2 = anduri viga, iga muu väärtus
`seat_state`	`uint8`	0 = tühi, 1 = hõivatud	2 = anduri viga, iga muu väärtus
`speed_raw`	`uint16`	0…30000 (1 bitt = 0,01 km/h, seega 0…300 km/h)	0xFFFF = puudub, üle 30000 = ebausutav
`age_ms`	`uint32`	0…200 ms	üle 200 ms = aegunud

Väljundid:

Väljund	Tähendus
`visual_on`	Märgutuli või teade näidikul (TRUE = sees)
`audio_on`	Helisignaal (TRUE = sees)
`state`	Koondolek: `NO_WARNING`, `VISUAL`, `VISUAL_AUDIO` või `DEGRADED` (veaolek)
`faults`	Vigade bitimask diagnostikaks: millised signaalid olid vigased

Miks `speed_raw` on täisarv, mitte km/h ujukomaarvuna: täisarvuline arvutus on täpne ja ettearvatav. 10 km/h lävi on seega väärtus 1000.

Otsustusloogika



otsustusloogika · 3 kontrolli, 4 tulemust

Kontrollid tehakse alati selles järjekorras: kõigepealt vead, siis hoiatuse vajadus, siis kiirus. Joonis vastab funktsioonile `Sbr_SelectState`; vigu tuvastab enne seda `Sbr_CheckSignals`.

Std_Types.h – põhitüübid

See fail annab AUTOSAR-is tavapärased tüübinimed, et kood näeks välja nagu päris autotarkvara ka ilma AUTOSAR-i tarkvarapakita.

```
#ifndef STD_TYPES_H
#define STD_TYPES_H

#include <stdint.h>
#include <stdbool.h>

typedef uint8_t  uint8;
typedef uint16_t uint16;
typedef uint32_t uint32;
typedef bool     boolean;

#ifndef TRUE
#define TRUE  (true)
#endif
#ifndef FALSE
#define FALSE (false)
#endif

typedef uint8 Std_ReturnType;
#define E_OK      ((Std_ReturnType)0x00U)
#define E_NOT_OK  ((Std_ReturnType)0x01U)

#ifndef NULL_PTR
#define NULL_PTR  ((void *)0)
#endif

#endif /* STD_TYPES_H */
```

Mida iga osa teeb:

- `#ifndef STD_TYPES_H` … `#endif` – *include guard*: kui faili kaasatakse mitu korda, loetakse see sisse ainult üks kord (MISRA Dir 4.10).
- `#include <stdint.h>` – annab täpse laiusega tüübid (`uint8_t` = alati 8 bitti). Tavaline `int` võib eri protsessoritel olla 16 või 32 bitti, seetõttu MISRA (Dir 4.6) seda ei soovita.
- `typedef uint8_t uint8;` jne – AUTOSAR-i nimed samadele tüüpidele.
- `typedef bool boolean;` – tõeväärtus. Kasutame C99 `bool` -i, sest MISRA tüübimudel tunneb selle ära kui tõeväärtuse (reeglid 10.1 ja 14.4).
- `Std_ReturnType` , `E_OK` , `E_NOT_OK` – AUTOSAR-i standardne funktsiooni tagastusväärtus: õnnestus / ei õnnestunud.

- `U` arvu lõpus (`0x00U`) – märgib, et konstant on märgita (MISRA reegel 7.2).
- `NULL_PTR` – AUTOSAR-i nimi tühjale viidale.

Sbr_Cfg.h – seadistus

Kõik arvud, mida võib tulevikus muuta, on ühes kohas. Loogikas endas pole ühtegi „maagilist arvu“.

```
#ifndef SBR_CFG_H
#define SBR_CFG_H

#include "Std_Types.h"

/* speed_raw scaling: 1 bit = 0.01 km/h (integer only, no floating point) */

/** REQ-SBR-002: audio warning only above 10 km/h (10.00 km/h = 1000 raw). */
#define SBR_SPEED_AUDIO_THRESHOLD_RAW  ((uint16)1000U)

/** Highest plausible speed: 300.00 km/h. Anything above is a fault. */
#define SBR_SPEED_RAW_MAX_VALID        ((uint16)30000U)

/** Raw value sent by the sender when speed is not available. */
#define SBR_SPEED_RAW_NOT_AVAILABLE    ((uint16)0xFFFFU)

/** REQ-SBR-003: input message older than this is treated as a fault. */
#define SBR_MSG_TIMEOUT_MS             ((uint32)200U)

#endif /* SBR_CFG_H */
```

Mida iga rida teeb:

- `SBR_SPEED_AUDIO_THRESHOLD_RAW = 1000` – helisignaali lävi. 1000 × 0,01 km/h = 10 km/h (REQ-SBR-002).
- `SBR_SPEED_RAW_MAX_VALID = 30000` – suurim usutav kiirus, 300 km/h. Suurem väärtus tähendab rikkis andurit või sõnumit.
- `SBR_SPEED_RAW_NOT_AVAILABLE = 0xFFFF` – väärtus, mille saatja paneb sõnumisse, kui kiirust pole teada. Siinisignaalidel on tavaks märkida puuduv väärtus kõik bitid üheks (AUTOSAR COM *invalid value*).
- `SBR_MSG_TIMEOUT_MS = 200` – sõnumi vanuse piir (REQ-SBR-003).
- `((uint16)1000U)` – konstandil on kohe õige tüüp. Siis ei teki võrdlusel varjatud tüübiteisendust (MISRA reeglid 10.3 ja 10.4).

AUTOSAR-is on seadistus alati eraldi failis (`<Moodul>_Cfg.h`). Nii saab sama koodi kasutada eri autodes, muutes ainult seda faili.

Sbr.h – mooduli liides

See fail ütleb teistele moodulitele, kuidas turvavöö moodulit kasutada: mis sisendid on, mis väljundid tulevad ja mis funktsiooni kutsuda. Faili alguses on kommentaarina kirjas ka nõuded REQ-SBR-001…003 ja ohutu oleku määratlus.

1. osa: sisendite koodid

```
/* ignition_on */
#define SBR_IGN_OFF           ((uint8)0U)
#define SBR_IGN_ON            ((uint8)1U)

/* belt_state */
#define SBR_BELT_UNBUCKLED    ((uint8)0U)
#define SBR_BELT_BUCKLED      ((uint8)1U)
#define SBR_BELT_FAULT        ((uint8)2U)   /* sensor reports a fault */

/* seat_state */
#define SBR_SEAT_EMPTY        ((uint8)0U)
#define SBR_SEAT_OCCUPIED     ((uint8)1U)
#define SBR_SEAT_FAULT        ((uint8)2U)   /* sensor reports a fault */
```

Iga sisendi väärtus saab nime. Koodis kirjutame `SBR_BELT_BUCKLED` , mitte `1` , nii on kood loetav ja vigu on raskem teha. Kood ei kasuta siin `enum` -i, sest siinilt tulev bait võib sisaldada mis tahes väärtust 0…255. Seda tuleb enne kontrollida, alles siis võib väärtust usaldada.

2. osa: vigade bitid

```
#define SBR_FAULT_NONE        ((uint8)0x00U)
#define SBR_FAULT_IGNITION    ((uint8)0x01U)
#define SBR_FAULT_BELT        ((uint8)0x02U)
#define SBR_FAULT_SEAT        ((uint8)0x04U)
#define SBR_FAULT_SPEED       ((uint8)0x08U)
#define SBR_FAULT_TIMEOUT     ((uint8)0x10U)
#define SBR_FAULT_NULL_INPUT  ((uint8)0x20U)
```

Igal vealõigil on oma bitt. Kui vigu on mitu, liidetakse bitid kokku: näiteks vöö viga + aegunud sõnum = 0x02 | 0x10 = 0x12. Diagnostikatööriist (nt töökojas) näeb sellest kohe, mis rikkis on.

3. osa: olekud ja andmestruktuurid

```
typedef enum
{
    SBR_STATE_NO_WARNING   = 0,  /* no warning needed                      */
    SBR_STATE_VISUAL       = 1,  /* visual warning only                    */
    SBR_STATE_VISUAL_AUDIO = 2,  /* visual + audio warning                 */
    SBR_STATE_DEGRADED     = 3   /* signal fault -> safe state (visual)    */
} Sbr_StateType;

typedef struct
{
    uint8  ignition_on;  /* SBR_IGN_xxx                                    */
    uint8  belt_state;   /* SBR_BELT_xxx                                   */
    uint8  seat_state;   /* SBR_SEAT_xxx                                   */
    uint16 speed_raw;    /* 0.01 km/h per bit, 0xFFFF = not available      */
    uint32 age_ms;       /* time since the message was received [ms]       */
} Sbr_InputType;

typedef struct
{
    boolean       visual_on;   /* telltale lamp / display message          */
    boolean       audio_on;    /* chime / buzzer                           */
    Sbr_StateType state;       /* overall state                            */
    uint8         faults;      /* SBR_FAULT_xxx bit mask                   */
} Sbr_OutputType;
```

- `Sbr_StateType` – neli võimalikku tulemust. Siin sobib `enum`, sest väärtuse arvutab moodul ise, see ei tule siinilt.
- `Sbr_InputType` – kõik viis sisendit ühes struktuuris. Väljade nimed on täpselt need, mis ülesandes.
- `Sbr_OutputType` – mida moodul väljastab: tuli, heli, olek ja vigade mask.
- Tüübinimed lõpevad `Type`-iga – AUTOSAR-i nimetamistava.

4. osa: funktsiooni deklaratsioon

```
Std_ReturnType Sbr_Evaluate(const Sbr_InputType *input, Sbr_OutputType
*output);
```

- See on mooduli ainus avalik funktsioon. Seda kutsutakse tsükliliselt, nt iga 10 ms järel.
- `const Sbr_InputType *input` – `const` lubab sisendeid lugeda, aga mitte muuta (MISRA reegel 8.13).
- Sisendid ja väljundid antakse viidaga. Moodulis pole ühtegi globaalset muutujat, seega on iga väljakutse sõltumatu.

Sbr.c 1/4 – Sbr_CheckSignals: vigade tuvastus

`Sbr.c` algab kaasatud failide ja sisemiste funktsioonide prototüüpidega:

```
#include "Sbr.h"
#include "Sbr_Cfg.h"

static uint8 Sbr_CheckSignals(const Sbr_InputType *input);
static Sbr_StateType Sbr_SelectState(const Sbr_InputType *input, uint8
faults);
static void Sbr_WriteOutput(Sbr_StateType state, uint8 faults,
                            Sbr_OutputType *output);
```

- `static` – funktsioon on nähtav ainult selles failis. Teised moodulid ei saa seda kogemata kutsuda (MISRA reegel 8.8).
- Prototüübid faili alguses – kompilaator kontrollib iga väljakutse parameetrite tüüpe (MISRA reegel 8.2).

Esimene funktsioon kontrollib, kas sisendeid saab usaldada. See rakendab reeglit REQ-SBR-003.

```
static uint8 Sbr_CheckSignals(const Sbr_InputType *input)
{
    uint8 faults = SBR_FAULT_NONE;

    /* Message older than 200 ms -> data cannot be trusted. */
    if (input->age_ms > SBR_MSG_TIMEOUT_MS)
    {
        faults |= SBR_FAULT_TIMEOUT;
    }

    /* Only the defined codes are valid; anything else is a fault. */
    switch (input->ignition_on)
    {
        case SBR_IGN_OFF:
        case SBR_IGN_ON:
            break;

        default:
            faults |= SBR_FAULT_IGNITION;
            break;
    }

    switch (input->belt_state)
    {
        case SBR_BELT_UNBUCKLED:
```

```
        case SBR_BELT_BUCKLED:
            break;

        case SBR_BELT_FAULT:
        default:
            faults |= SBR_FAULT_BELT;
            break;
    }

    switch (input->seat_state)
    {
        case SBR_SEAT_EMPTY:
        case SBR_SEAT_OCCUPIED:
            break;

        case SBR_SEAT_FAULT:
        default:
            faults |= SBR_FAULT_SEAT;
            break;
    }

    /* Speed: "not available" marker or physically implausible value. */
    if ((input->speed_raw == SBR_SPEED_RAW_NOT_AVAILABLE) ||
        (input->speed_raw > SBR_SPEED_RAW_MAX_VALID))
    {
        faults |= SBR_FAULT_SPEED;
    }

    return faults;
}
```

Samm-sammult:

1. `uint8 faults = SBR_FAULT_NONE;` – alguses vigu pole. Muutuja saab väärtuse kohe deklareerimisel (MISRA reegel 9.1).
2. **Sõnumi vanus.** Kui `age_ms` on üle 200, pannakse viga `SBR_FAULT_TIMEOUT`. Täpselt 200 ms on veel lubatud, sest reegel ütleb „vanem kui 200 ms“.
3. **Süüde.** `switch` laseb läbi ainult väärtused 0 ja 1. Kõik muu (2…255) läheb `default`-harusse ja on viga.
4. **Turvavöö ja iste.** Sama põhimõte. Anduri enda teatatud viga (`SBR_BELT_FAULT`, `SBR_SEAT_FAULT`) ja tundmatu kood käsitletakse ühtemoodi.
5. **Kiirus.** Viga on, kui väärtus on 0xFFFF (puudub) või üle 30000 (üle 300 km/h, füüsikaliselt ebausutav).

6. `faults |= ...` – lisab biti maskile, teisi bitte muutmata. Nii kogutakse kõik vead korraga.
7. Funktsioon tagastab vigade maski. 0 tähendab, et kõik sisendid on korras.

Miks nii: ISO 26262 nõuab, et tarkvara tuvastaks vigased sisendid ega teeks nende põhjal otsuseid. Iga `switch` lõpeb `default`-iga ja iga haru `break`-iga (MISRA reeglid 16.3 ja 16.4). Iga `if` sisu on looksulgudes, ka üherealine (reegel 15.6).

Sbr.c 2/4 – Sbr_SelectState: hoiatuse valik

Teine funktsioon otsustab, milline hoiatus on vaja. See rakendab kõiki kolme reeglit kindlas tähtsuse järjekorras.

```
static Sbr_StateType Sbr_SelectState(const Sbr_InputType *input, uint8
faults)
{
    Sbr_StateType state;

    if (faults != SBR_FAULT_NONE)
    {
        /* REQ-SBR-003: faulty or stale signal -> visual warning (safe
state).
         * Audio stays off because the warning conditions cannot be
         * confirmed from faulty data. */
        state = SBR_STATE_DEGRADED;
    }
    else if ((input->ignition_on == SBR_IGN_ON) &&
             (input->seat_state == SBR_SEAT_OCCUPIED) &&
             (input->belt_state == SBR_BELT_UNBUCKLED))
    {
        /* REQ-SBR-001: warning needed. */
        if (input->speed_raw > SBR_SPEED_AUDIO_THRESHOLD_RAW)
        {
            /* REQ-SBR-002: above 10 km/h -> visual + audio. */
            state = SBR_STATE_VISUAL_AUDIO;
        }
        else
        {
            /* REQ-SBR-002: always at least visual. */
            state = SBR_STATE_VISUAL;
        }
    }
    else
    {
        /* REQ-SBR-001: ignition off, seat empty or belt fastened. */
```

```
        state = SBR_STATE_NO_WARNING;
    }

    return state;
}
```

Samm-sammult:

1. **Kas on vigu?** (REQ-SBR-003) Kui mask pole 0, on tulemus `SBR_STATE_DEGRADED`: märgutuli põleb, heli ei kõla. See kontroll on esimene, sest vigaste andmete põhjal ei tohi edasi otsustada.
2. **Kas hoiatus on vajalik?** (REQ-SBR-001) Kõik kolm tingimust peavad korraga kehtima: süüde sees **ja** iste hõivatud **ja** vöö kinnitamata.
3. **Kas lisada heli?** (REQ-SBR-002) Kui kiirus on üle 1000 (10,00 km/h), on tulemus `VISUAL_AUDIO`. Muidu ainult `VISUAL`. Täpselt 10,00 km/h juures heli ei kõla, sest reegel ütleb „üle 10 km/h“.
4. **Muul juhul** pole hoiatust vaja (`NO_WARNING`).

Miks nii:

- `if … else if … else` ahel lõpeb alati `else`-ga, nii on iga võimalik juhtum kaetud (MISRA reegel 15.7).
- Iga tingimus on eraldi sulgudes, et tehete järjekord oleks selge (MISRA reegel 12.1).
- Funktsioonil on üks `return` lõpus (MISRA reegel 15.5). Nii on lihtsam jälgida, mis väärtus välja läheb.
- Veaolekus heli ei kõla: reegel 3 nõuab ainult visuaalset hoiatust ja vigaste andmetega ei saa kinnitada, et vöö on lahti. Nii väldime valehoiatusi.

Sbr.c 3/4 – Sbr_WriteOutput: väljundite kirjutamine

Kolmas funktsioon teisendab oleku konkreetseteks käskudeks: kas tuli põleb ja kas heli kõlab.

```
static void Sbr_WriteOutput(Sbr_StateType state, uint8 faults,
                            Sbr_OutputType *output)
{
    switch (state)
    {
        case SBR_STATE_NO_WARNING:
            output->visual_on = FALSE;
            output->audio_on  = FALSE;
            output->state     = SBR_STATE_NO_WARNING;
            break;
```

```
        case SBR_STATE_VISUAL:
            output->visual_on = TRUE;
            output->audio_on  = FALSE;
            output->state     = SBR_STATE_VISUAL;
            break;

        case SBR_STATE_VISUAL_AUDIO:
            output->visual_on = TRUE;
            output->audio_on  = TRUE;
            output->state     = SBR_STATE_VISUAL_AUDIO;
            break;

        case SBR_STATE_DEGRADED:
        default:
            /* Safe state: visual ON, audio OFF. */
            output->visual_on = TRUE;
            output->audio_on  = FALSE;
            output->state     = SBR_STATE_DEGRADED;
            break;
    }

    output->faults = faults;
}
```

Olek	`visual_on`	`audio_on`
`NO_WARNING`	FALSE	FALSE
`VISUAL`	TRUE	FALSE
`VISUAL_AUDIO`	TRUE	TRUE
`DEGRADED` (ja iga tundmatu väärtus)	TRUE	FALSE

Miks nii:

- Igas harus kirjutatakse kõik väljad. Ükski väljund ei jää eelmisest väljakutsest vana väärtusega.
- `default` on koos `SBR_STATE_DEGRADED`-iga. Kui olekumuutuja peaks mälus rikutuks minema (nt kiirgus või mäluviga), satub tarkvara ohutusse olekusse. See on *kaitsev programmeerimine* (ISO 26262) ja MISRA reegel 16.4.
- Vigade mask kopeeritakse väljundisse alati, et diagnostika näeks põhjust.

Sbr.c 4/4 – Sbr_Evaluate: avalik funktsioon

See on ainus funktsioon, mida teised moodulid kutsuvad. See paneb kolm eelmist funktsiooni kokku ja kontrollib viitu.

```
Std_ReturnType Sbr_Evaluate(const Sbr_InputType *input, Sbr_OutputType
*output)
{
    Std_ReturnType ret    = E_NOT_OK;
    Sbr_StateType  state  = SBR_STATE_DEGRADED;    /* safe default */
    uint8          faults = SBR_FAULT_NULL_INPUT;  /* safe default */

    if (input != NULL_PTR)
    {
        faults = Sbr_CheckSignals(input);
        state  = Sbr_SelectState(input, faults);
        ret    = E_OK;
    }

    if (output != NULL_PTR)
    {
        Sbr_WriteOutput(state, faults, output);
    }
    else
    {
        ret = E_NOT_OK;
    }

    return ret;
}
```

Samm-sammult:

1. **Ohutud algväärtused.** Muutujad saavad alguses ohutu oleku väärtused: `DEGRADED` ja viga „sisend puudub“. Kui midagi läheb valesti, on tulemus vaikimisi ohutu.
2. **Sisendi viida kontroll.** Kui `input` on olemas, kontrollitakse signaale (`Sbr_CheckSignals`) ja valitakse olek (`Sbr_SelectState`). Kui `input` on `NULL_PTR` , jääb ohutu olek alles.
3. **Väljundi viida kontroll.** Kirjutame ainult siis, kui `output` on olemas. Tühja viida kaudu kirjutamine kõrvaldaks programmi.
4. **Tagastusväärtus.** `E_OK` ainult siis, kui mõlemad viidad olid korras. Muidu `E_NOT_OK` , et kutsuja teaks probleemist.

Olukord	Tagastus	Mis väljundisse läheb
Mõlemad viidad korras	`E_OK`	Arvutatud hoiatus
`input` = `NULL_PTR`	`E_NOT_OK`	Ohutu olek: tuli põleb, heli ei kõla
`output` = `NULL_PTR`	`E_NOT_OK`	Mitte midagi (pole kuhu kirjutada)

Miks nii: AUTOSAR-i moodulid kontrollivad alati viidaparameetreid ja tagastavad `E_NOT_OK`. Funktsioon ei kasuta globaalseid muutujaid, dünaamilist mälu ega rekursiooni. Seetõttu on ta *deterministlik*: samad sisendid annavad alati sama tulemuse (ISO 26262-6, tabel 6).

Kasutamise näide

Juht istub autos, süüde on sees, vöö on kinnitamata ja auto sõidab 15 km/h. Tulemus: tuli põleb ja heli kõlab.

```
#include "Sbr.h"

void Cluster_MainFunction_10ms(void)
{
    Sbr_InputType  in;
    Sbr_OutputType out;

    in.ignition_on = SBR_IGN_ON;          /* süüde sees          */
    in.belt_state  = SBR_BELT_UNBUCKLED;  /* vöö kinnitamata      */
    in.seat_state  = SBR_SEAT_OCCUPIED;   /* iste hõivatud        */
    in.speed_raw   = 1500U;               /* 15,00 km/h           */
    in.age_ms      = 20U;                 /* sõnum 20 ms vana     */

    if (Sbr_Evaluate(&in, &out) == E_OK)
    {
        /* out.visual_on == TRUE, out.audio_on == TRUE,
           out.state == SBR_STATE_VISUAL_AUDIO */
    }
}
```

Päris autos täidetakse `in` väljad siinilt saabunud sõnumist ja `out` väljad saadetakse näidikule ja helisignaali juhtimisele. Funktsiooni kutsutakse tsükliliselt, nt iga 10 ms järel.

Standardid: mida ja kus rakendati

Kolm standardit täidavad eri rolli. **MISRA C** ütleb, kuidas C-koodi ohutult kirjutada. **AUTOSAR** annab autotööstuse ühtsed nimed, tüübid ja moodulite ülesehituse. **ISO 26262** nõuab, et tarkvara tuvastaks vigu, läheks vea korral ohutusse olekusse ja oleks tõendatult testitud.

Standard	Nõue või soovitus	Kus koodis
MISRA C:2012 Dir 4.6	Täpse laiusega tüübid, mitte `int` / `char`	`Std_Types.h`, kõik muutujad
MISRA C:2012 7.2	Märgita konstantidel `U` sufiks	kõik konstandid
MISRA C:2012 8.2, 8.8	Prototüübid; sisemised funktsioonid `static`	`Sbr.c` algus
MISRA C:2012 8.13	Viit, mille kaudu ei muudeta, on `const`	`Sbr_Evaluate`, `Sbr_CheckSignals`
MISRA C:2012 9.1	Muutuja saab väärtuse enne kasutamist	`Sbr_Evaluate`, `Sbr_CheckSignals`
MISRA C:2012 10.1, 14.4	Tingimus on tõeväärtus (`boolean` = `bool`)	`Std_Types.h`
MISRA C:2012 15.5	Üks väljumispunkt funktsioonis	kõik funktsioonid
MISRA C:2012 15.6, 15.7	Looksulud igal harul; `else if` ahel lõpeb `else` -ga	`Sbr_SelectState`
MISRA C:2012 16.3, 16.4	Iga `switch` haru lõpeb `break` -iga; `default` on alati olemas	`Sbr_CheckSignals`, `Sbr_WriteOutput`
MISRA C:2012 17.2, 21.3, 21.6	Rekursiooni, dünaamilist mälu ja `stdio.h` -d ei kasutata	kogu moodul
AUTOSAR Platform/Std Types	`uint8`, `boolean`, `Std_ReturnType`, `E_OK` / `E_NOT_OK`, `NULL_PTR`	`Std_Types.h`
AUTOSAR nimetamistava	Mooduli eesliide (`Sbr_`), tüübid lõpevad `Type` -iga	kõik nimed
AUTOSAR konfiguratsioon	Seadistus eraldi failis	`Sbr_Cfg.h`
AUTOSAR viidakontroll	`NULL_PTR` argumendil tagastatakse `E_NOT_OK`	`Sbr_Evaluate`

Standard	Nõue või soovitus	Kus koodis
AUTOSAR COM / E2E	Signaali *invalid value* (0xFF, 0xFFFF); sõnumi värskuse kontroll	`Sbr_CheckSignals`
ISO 26262 ohutu olek	Vea korral tuli põleb, heli ei kõla	`Sbr_SelectState`, `Sbr_WriteOutput`, `Sbr_Evaluate`
ISO 26262 vigade tuvastus	Iga sisendi vahemik + ajalimiit + vigade mask	`Sbr_CheckSignals`
ISO 26262-6 tabel 6	Globaalseid muutujaid, dünaamilisi objekte ega kaudseid tüübiteisendusi pole	kogu moodul
ISO 26262 jälgitavus	Nõuded `REQ-SBR-xxx` koodis ja testides	`Sbr.h`, `Sbr.c`, `test_Sbr.c`
ISO 26262-6 tabelid 7–9	Nõuetepõhised testid, piirväärtused, vigade sisestamine, katvus	`test_Sbr.c`

Kontroll tööriistaga: `cppcheck --addon=misra` andis **0 leidu**. cppcheck ei kata kõiki MISRA reegleid. Tööstuses kasutatakse sertifitseeritud tööriistu, nt Polyspace, PC-lint Plus või LDRA.

Testid ja tulemused

Kõik 34 kontrolli läbisid. `Sbr.c` ridadest ja harudest on testidega läbitud 100%.

Testiosa	Mida kontrollib	Kontrolle
1. Nõuetepõhised testid	Iga reegel eraldi, kõik 8 süüte/iste/vöö kombinatsiooni, piirväärtused, iga vigane signaal	29
2. API testid	`NULL_PTR` sisend, `NULL_PTR` väljund, mõlemad `NULL_PTR`, korrektne kutse	4
3. *Back-to-back* test	~170 miljonit sisendikombinatsiooni võrreldakse eraldi kirjutatud referentsmudeliga	1 (0 erinevust)

1. Nõuetepõhised testid on tabelina. Igal real on nõude tunnus, nimi, sisendid ja oodatav tulemus:

```
/* req, nimi, { süüde, vöö, iste, kiirus, vanus }, visual, audio, olek, vead
*/
{ "REQ-SBR-002", "10.00 km/h -> visual only",    { ON, UNB, OCC, 1000U, 0U },
TRUE, FALSE, S_VIS, SBR_FAULT_NONE },
{ "REQ-SBR-002", "10.01 km/h -> visual + audio", { ON, UNB, OCC, 1001U, 0U },
TRUE, TRUE,  S_AUD, SBR_FAULT_NONE },
{ "REQ-SBR-003", "age 200 ms -> still valid",    { ON, UNB, OCC, 5000U, 200U
}, TRUE, TRUE,  S_AUD, SBR_FAULT_NONE },
{ "REQ-SBR-003", "age 201 ms -> safe state",     { ON, UNB, OCC, 5000U, 201U
}, TRUE, FALSE, S_DEG, SBR_FAULT_TIMEOUT },
```

Piirväärtused on testitud mõlemalt poolt (ISO 26262-6 tabel 8): 9,99 / 10,00 / 10,01 km/h ja 200 / 201 ms. Just piiridel tekivad kõige sagedamini vead, näiteks `>` asemel `>=`.

3. Back-to-back test võrdleb moodulit lihtsa referentsmudeliga, mis on kirjutatud otse ülesande tekstist:

```
static void Reference(const Sbr_InputType *in, boolean *visual, boolean
*audio)
{
    boolean ign_ok   = (in->ignition_on <= 1U);
    boolean belt_ok  = (in->belt_state  <= 1U);
    boolean seat_ok  = (in->seat_state  <= 1U);
    boolean speed_ok = (in->speed_raw   <= 30000U);
    boolean fresh    = (in->age_ms      <= 200U);

    if (!(ign_ok && belt_ok && seat_ok && speed_ok && fresh))
    {
        *visual = TRUE;            /* rule 3 */
        *audio  = FALSE;
    }
    else if ((in->ignition_on == 1U) && (in->seat_state == 1U) && (in-
>belt_state == 0U))
    {
        *visual = TRUE;            /* rules 1 + 2 */
        *audio  = (in->speed_raw > 1000U);
    }
    else
    {
        *visual = FALSE;
        *audio  = FALSE;
    }
}
```

Test käib läbi kõik 256 × 256 × 256 süüte/vöö/istme koodi ja kõik 65 536 kiiruse väärtust. Iga kombinatsiooni korral peab moodul andma sama tulemuse kui referents. Lisaks kontrollitakse, et heli ei kõla kunagi ilma märgutuleta.

Tulemus:

```
3) Back-to-back test vs. reference model
   169869312 input combinations compared, 0 mismatches

Summary: 34 checks, 0 failed -> ALL PASSED

File 'Sbr.c'
Lines executed:100.00% of 73
Branches executed:100.00% of 30
Taken at least once:100.00% of 30

MISRA: no findings
```

MC/DC katvust (iga tingimuse mõju eraldi) ei mõõdetud, sest kasutatud gcc versioon seda ei toeta.

Kompileerimine (Makefile)

Koodi kompileerimiseks ja testimiseks piisab neljäst käsust:

```
make test       # kompileerib rangete hoiatustega ja käivitab testid
make coverage   # mõõdab rea- ja harukatvust (gcov)
make misra      # MISRA C:2012 kontroll (cppcheck)
make clean      # kustutab kompileeritud failid
```

Kompilaatori lipud failis `Makefile`:

```
CFLAGS := -std=c99 -O2 -Wall -Wextra -Wpedantic -Wconversion -Wsign-
conversion \
          -Wshadow -Wcast-qual -Wstrict-prototypes -Wmissing-prototypes \
          -Wswitch-default -Wundef -Werror
```

Lipp	Mida teeb
`-std=c99`	Kasutab C99 standardit (MISRA C:2012 toetab C90 ja C99)
`-Wall -Wextra -Wpedantic`	Lülitab sisse kõik tavalised hoiatused ja standardist kõrvalekalded

Lipp	Mida teeb
`-Wconversion -Wsign-conversion`	Hoiatab varjatud tüübiteisenduste eest (MISRA 10. peatükk)
`-Wshadow`	Hoiatab, kui sisemine muutuja varjab välise sama nimega muutuja (MISRA 5.3)
`-Wswitch-default`	Hoiatab, kui `switch`-il puudub `default` (MISRA 16.4)
`-Werror`	Iga hoiatus on viga: kood ei kompileeru enne, kui kõik hoiatused on parandatud

Kood kompileerub nende lippudega gcc 13 ja clang-iga ilma ühegi hoiatuseta. Kompileerimiseks ilma Makefile'ita:

```
gcc -std=c99 -Wall -Wextra -o test_Sbr Sbr.c test_Sbr.c
./test_Sbr
```

Käivitamine VS Code'is

Kaustas `.vscode` on valmis seadistus: **Ctrl+Shift+B** kompileerib ja käivitab testid, **F5** käivitab siluri. Testid jäuavad lõpuni umbes 4 sekundiga.

1. Paigalda C-kompilaator:
   - **Windows:** paigalda MSYS2. Ava MSYS2 UCRT64 terminal ja käivita `pacman -S --needed mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-gdb`. Lisa `C:\msys64\ucrt64\bin` Windowsi PATH-i ja taaskäivita VS Code.
   - **Mac:** terminalis `xcode-select --install`.
   - **Linux (Ubuntu):** `sudo apt install build-essential gdb`.
2. Paigalda VS Code'i laiendus **C/C++** (Microsoft). VS Code pakub seda kausta avamisel ise.
3. Paki `turvavoo.zip` lahti ja ava kaust `turvavoo`: **File → Open Folder**.
4. Vajuta **Ctrl+Shift+B** (Macis Cmd+Shift+B). Kood kompileeritakse ja testide tulemus ilmub terminali.
5. Silumiseks kliki rea numbri ees (tekib punane murdepunkt), nt `Sbr.c` funktsioonis `Sbr_SelectState`, ja vajuta **F5**.

Kontrolli, et kompilaator on olemas: VS Code'i terminalis (**Terminal → New Terminal**) peab `gcc --version` näitama versiooni.

Ära kasuta nuppu „Run C/C++ File“ (▷). See kompileerib ainult avatud faili ja annab vea `undefined reference to Sbr_Evaluate`. Programm vajab mõlemat faili: `Sbr.c` ja `test_Sbr.c`.

Ilma seadistuseta saab sama teha VS Code'i terminalis:

```
gcc -std=c99 -Wall -Wextra -o test_Sbr Sbr.c test_Sbr.c
./test_Sbr          # Windowsis: .\test_Sbr.exe
```

`.vscode/tasks.json` – mida Ctrl+Shift+B teeb:

```
{
    // VS Code'i ülesanded: Ctrl+Shift+B kompileerib ja käivitab testid.
    "version": "2.0.0",
    "tasks": [
        {
            // Kompileerib mõlemad .c failid üheks programmiks (silumisinfoga
-g).
            "label": "Kompileeri",
            "type": "shell",
            "command": "gcc",
            "args": [
                "-std=c99", "-g", "-O0",
                "-Wall", "-Wextra", "-Wpedantic",
                "-o", "test_Sbr",
                "Sbr.c", "test_Sbr.c"
            ],
            "options": { "cwd": "${workspaceFolder}" },
            "problemMatcher": ["$gcc"]
        },
        {
            // Kompileerib ja käivitab testid. Vaikimisi ülesanne:
Ctrl+Shift+B.
            "label": "Kompileeri ja käivita",
            "type": "shell",
            "command": "./test_Sbr",
            "windows": { "command": ".\\test_Sbr.exe" },
            "options": { "cwd": "${workspaceFolder}" },
            "dependsOn": "Kompileeri",
            "group": { "kind": "build", "isDefault": true },
            "problemMatcher": []
        }
    ]
}
```

`.vscode/launch.json` – mida F5 teeb:

```
{
    // Silumine: F5 kompileerib programmi ja käivitab selle siluriga.
    // Murdepunkti paneku jaoks kliki rea numbri ees (nt Sbr.c, funktsioon
Sbr_SelectState).
    "version": "0.2.0",
    "configurations": [
        {
            "name": "Silu testprogrammi",
            "type": "cppdbg",
            "request": "launch",
            "program": "${workspaceFolder}/test_Sbr",
            "args": [],
            "cwd": "${workspaceFolder}",
            "stopAtEntry": false,
            "externalConsole": false,
            "MIMode": "gdb",
            "preLaunchTask": "Kompileeri",
            "windows": {
                "program": "${workspaceFolder}/test_Sbr.exe",
                "miDebuggerPath": "C:\\msys64\\ucrt64\\bin\\gdb.exe"
            },
            "osx": {
                "MIMode": "lldb"
            }
        }
    ]
}
```

Kaustas on veel `c_cpp_properties.json` (IntelliSense kasutab C99 standardit) ja `extensions.json` (soovitab laiendust C/C++).

Eeldused ja disainiotsused

Ülesanne jätab mõned asjad lahtiseks. Need otsused on tehtud ja neid saab vajadusel muuta.

Teema	Otsus	Miks
„Üle 10 km/h“	Rangelt suurem kui 10,00 km/h; täpselt 10,00 annab ainult visuaalse hoiatuse	Reegli sõnastus „üle“
„Vanem kui 200 ms“	Viga alates 201 ms; täpselt 200 ms on veel kehtiv	Reegli sõnastus „vanem kui“

Teema	Otsus	Miks
Heli veaolekus	Väljas	Reegel 3 nõuab ainult visuaalset; vigaste andmetega ei saa kinnitada, et vöö on lahti. Muudetav funktsioonis `Sbr_SelectState`.
Reegli 3 prioriteet	Kõrgeim, ka siis, kui süüde on väljas	Reegel ütleb „kui mõni signaal on vigane“ ilma lisatingimuseta
Kiiruse skaala	1 bitt = 0,01 km/h, piir 300 km/h	Levinud CAN-signaali skaala; täisarvuline arvutus
Sisendite koodid	0/1 kehtivad, 2 = anduri viga, ülejäänud = kehtetu	Ülesanne koode ei määra
ASIL tase	Eeldatud QM või ASIL A	Turvavöö meeldetuletus ei juhi sõidukit. Kõrgema ASIL-i korral lisanduksid nt E2E CRC, sõnumiloendur ja riistvaraline jälgimine.
Koodi kommentaarid	Eesti keeles	Kursusetöö jaoks loetavam. Failid on UTF-8 kodeeringus ja kompileeruvad gcc ning clang-iga hoiatusteta. Programmi väljundtekstid on inglise keeles, et tabeli joondus täpitähtede tõttu ei nihkuks.

Allikad

- ISO 26262-6:2018 – Road vehicles – Functional safety – Part 6: Product development at the software level (ISO ametlik leht)
- MISRA C (MISRA ametlik leht; uusim väljaanne on MISRA C:2023)
- MISRA C:2012 reeglite ja direktiivide loetelu lühikirjeldustega (MathWorks Polyspace)
- AUTOSAR CP SWS Standard Types, R23-11 (`Std_Types.h`, `Std_ReturnType`, `E_OK`)
- Cppcheck (staatilise analüüsi tööriist koos MISRA lisamooduliga)

ISO 26262 ja MISRA täistekstid on tasulised. Aadressid on leitud veebiotsingu kaudu.

Lisa: täielik lähtekood

All on kõik failid täies mahus, täpselt sellisena, nagu need kompileeriti ja testiti. Koodi kommentaarid selgitavad eesti keeles, mida iga osa teeb.

Std_Types.h

```
/**
 * @file    Std_Types.h
 * @brief   AUTOSAR-stiilis põhitüübid (lihtsustatud Std_Types.h ja
 *          Platform_Types.h), et moodul kompileeruks ka ilma AUTOSAR-i
 *          tarkvarapakita.
 *
 * Märkus: 'boolean' on C99 'bool', sest siis loeb MISRA C:2012 seda
 * tõeväärtustüübiks (reeglid 14.4 ja 10.1).
 */

/* Include guard: faili sisu loetakse sisse ainult üks kord, isegi kui
 * faili kaasatakse mitu korda (MISRA Dir 4.10). */
#ifndef STD_TYPES_H
#define STD_TYPES_H

#include <stdint.h>   /* täpse laiusega täisarvud: uint8_t, uint16_t,
uint32_t */
#include <stdbool.h>  /* C99 tõeväärtustüüp bool ning väärtused true ja false
*/

/* AUTOSAR-i nimed täpse laiusega tüüpidele. Tavalise 'int'-i suurus sõltub
 * protsessorist, need tüübid on alati sama laiusega (MISRA Dir 4.6). */
typedef uint8_t  uint8;    /* märgita täisarv, 8 bitti:  0 ... 255
*/
typedef uint16_t uint16;   /* märgita täisarv, 16 bitti: 0 ... 65 535
*/
typedef uint32_t uint32;   /* märgita täisarv, 32 bitti: 0 ... 4 294 967 295
*/
typedef bool     boolean;  /* tõeväärtus: TRUE või FALSE
*/

/* Tõeväärtuste konstandid (defineeritakse ainult siis, kui neid veel pole).
*/
#ifndef TRUE
#define TRUE  (true)
#endif
#ifndef FALSE
#define FALSE (false)
#endif
```

```
/* AUTOSAR-i standardne funktsiooni tagastustüüp. */
typedef uint8 Std_ReturnType;
#define E_OK      ((Std_ReturnType)0x00U)   /* funktsioon õnnestus      */
#define E_NOT_OK  ((Std_ReturnType)0x01U)   /* funktsioon ei õnnestunud */

/* Tühi viit (AUTOSAR-i nimi C-keele NULL-i jaoks). */
#ifndef NULL_PTR
#define NULL_PTR  ((void *)0)
#endif

#endif /* STD_TYPES_H */
```

Sbr_Cfg.h

```
/**
 * @file    Sbr_Cfg.h
 * @brief   Turvavöö meeldetuletuse (SBR) seadistus.
 *
 * Kõik seadistatavad arvud on siin failis. Nii pole loogikas endas
 * "maagilisi arve" (MISRA) ja seadistus on koodist eraldi (AUTOSAR).
 * Teise auto jaoks piisab ainult selle faili muutmisest.
 */
#ifndef SBR_CFG_H
#define SBR_CFG_H

#include "Std_Types.h"   /* uint16, uint32 */

/* speed_raw skaala: 1 bitt = 0,01 km/h.
 * Arvutame ainult täisarvudega, ujukomaarve ei kasutata. */

/** REQ-SBR-002: heli lisandub ainult üle 10 km/h (10,00 km/h = 1000). */
#define SBR_SPEED_AUDIO_THRESHOLD_RAW  ((uint16)1000U)

/** Suurim usutav kiirus: 300,00 km/h. Suurem väärtus loetakse veaks. */
#define SBR_SPEED_RAW_MAX_VALID        ((uint16)30000U)

/** Väärtus, mille saatja paneb sõnumisse, kui kiirust pole teada. */
#define SBR_SPEED_RAW_NOT_AVAILABLE    ((uint16)0xFFFFU)

/** REQ-SBR-003: sellest vanem sõnum [ms] loetakse veaks. */
#define SBR_MSG_TIMEOUT_MS             ((uint32)200U)

#endif /* SBR_CFG_H */
```

Sbr.h

```
/**
 * @file    Sbr.h
 * @brief   Turvavöö meeldetuletuse (SBR = Seat Belt Reminder) liides.
 *
 * Nõuded (ülesandest):
 *  REQ-SBR-001  Hoiatus on vajalik ainult siis, kui süüde on sees, iste on
 *               hõivatud ja turvavöö ei ole kinnitatud.
 *  REQ-SBR-002  Alati vähemalt visuaalne hoiatus. Kui kiirus on üle
 *               10 km/h, lisandub heli.
 *  REQ-SBR-003  Kui mõni signaal on vigane või sõnum vanem kui 200 ms,
 *               peab visuaalne hoiatus töötama (ohutu olek).
 *
 * Ohutus (ISO 26262):
 *  - Ohutu olek: visuaalne hoiatus SEES, heli VÄLJAS.
 *  - Iga sisendi väärtust kontrollitakse; tundmatu kood on viga.
 *  - Funktsioon ei mäleta midagi eelmisest korrast: samad sisendid
 *    annavad alati sama tulemuse (deterministlik).
 *  - Dünaamilist mälu, rekursiooni ega ujukomaarve ei kasutata.
 */
#ifndef SBR_H
#define SBR_H

#include "Std_Types.h"   /* uint8, uint16, uint32, boolean, Std_ReturnType */

/* ---------------------------------------------------------------------------
*/
/* Sisendite koodid (toorväärtused, nagu need tulevad sõiduki siinilt)
*/
/* ---------------------------------------------------------------------------
*/

/* ignition_on: süüte olek */
#define SBR_IGN_OFF           ((uint8)0U)   /* süüde väljas */
#define SBR_IGN_ON            ((uint8)1U)   /* süüde sees   */
/* iga muu väärtus = vigane või puuduv signaal */

/* belt_state: turvavöö lukk */
#define SBR_BELT_UNBUCKLED    ((uint8)0U)   /* vöö kinnitamata    */
#define SBR_BELT_BUCKLED      ((uint8)1U)   /* vöö kinnitatud     */
#define SBR_BELT_FAULT        ((uint8)2U)   /* andur teatab veast */
/* iga muu väärtus = vigane või puuduv signaal */

/* seat_state: istme andur */
#define SBR_SEAT_EMPTY        ((uint8)0U)   /* iste tühi          */
```

```
#define SBR_SEAT_OCCUPIED      ((uint8)1U)   /* iste hõivatud      */
#define SBR_SEAT_FAULT         ((uint8)2U)   /* andur teatab veast */
/* iga muu väärtus = vigane või puuduv signaal */

/* ----------------------------------------------------------------------
*/
/* Vigade bitid (bitimask diagnostika jaoks)
*/
/* Igal veal on oma bitt; mitu viga korraga = bitid kokku (nt 0x02 | 0x10).
*/
/* ----------------------------------------------------------------------
*/
#define SBR_FAULT_NONE         ((uint8)0x00U)   /* vigu pole
*/
#define SBR_FAULT_IGNITION     ((uint8)0x01U)   /* süüte signaal vigane
*/
#define SBR_FAULT_BELT         ((uint8)0x02U)   /* vöö signaal vigane
*/
#define SBR_FAULT_SEAT         ((uint8)0x04U)   /* istme signaal vigane
*/
#define SBR_FAULT_SPEED        ((uint8)0x08U)   /* kiirus puudub või vale
*/
#define SBR_FAULT_TIMEOUT      ((uint8)0x10U)   /* sõnum vanem kui 200 ms
*/
#define SBR_FAULT_NULL_INPUT   ((uint8)0x20U)   /* sisendi viit on NULL_PTR
*/

/* ----------------------------------------------------------------------
*/
/* Tüübid
*/
/* ----------------------------------------------------------------------
*/

/** Ühe hindamise koondtulemus. */
typedef enum
{
    SBR_STATE_NO_WARNING   = 0,  /* hoiatust pole vaja                 */
    SBR_STATE_VISUAL       = 1,  /* ainult visuaalne hoiatus (tuli)    */
    SBR_STATE_VISUAL_AUDIO = 2,  /* tuli + helisignaal                 */
    SBR_STATE_DEGRADED     = 3   /* signaali viga -> ohutu olek (tuli) */
} Sbr_StateType;

/** Sisendsignaalid (väljade nimed täpselt nagu ülesandes). */
typedef struct
{
```

```
    uint8  ignition_on;  /* süüde: SBR_IGN_xxx                              */
    uint8  belt_state;   /* turvavöö: SBR_BELT_xxx                          */
    uint8  seat_state;   /* iste: SBR_SEAT_xxx                              */
    uint16 speed_raw;    /* kiirus, 1 bitt = 0,01 km/h; 0xFFFF = puudub */
    uint32 age_ms;       /* aeg sõnumi saabumisest [ms]                     */
} Sbr_InputType;

/** Väljundkäsud näidikule ja helisignaalile. */
typedef struct
{
    boolean       visual_on;   /* märgutuli või teade näidikul: TRUE = sees
*/
    boolean       audio_on;    /* helisignaal: TRUE = sees
*/
    Sbr_StateType state;       /* koondolek
*/
    uint8         faults;      /* vigade bitimask (SBR_FAULT_xxx)
*/
} Sbr_OutputType;

/* ----------------------------------------------------------------------
*/
/* Avalik funktsioon (API)
*/
/* ----------------------------------------------------------------------
*/

/**
 * @brief  Arvutab ühe tsükli turvavöö hoiatuse.
 *
 * Kutsu seda tsükliliselt (nt iga 10 ms järel) värskete signaalidega.
 *
 * @param[in]  input   Sisendsignaalid. Kui NULL_PTR, kirjutatakse ohutu
olek.
 * @param[out] output  Hoiatuse käsud. Ei tohi olla NULL_PTR.
 *
 * @return E_OK      sisendid hinnati
 * @return E_NOT_OK  mõni argument oli NULL_PTR (kui output on olemas,
 *                   kirjutati sinna ohutu olek)
 */
Std_ReturnType Sbr_Evaluate(const Sbr_InputType *input, Sbr_OutputType
*output);

#endif /* SBR_H */
```

Sbr.c

```
/**
 * @file    Sbr.c
 * @brief   Turvavöö meeldetuletuse (SBR) loogika.
 *
 * Järgitud reeglid (lühidalt):
 *  - MISRA C:2012: täpse laiusega tüübid, U-sufiks märgita konstantidel,
 *    looksulud igal if/else harul, iga if-else-if ahel lõpeb else-ga,
 *    igal switch-il on default, igal funktsioonil üks väljumispunkt,
 *    pole dünaamilist mälu, rekursiooni, ujukomaarve ega stdio-d.
 *  - AUTOSAR: mooduli eesliide (Sbr_), Std_ReturnType, NULL_PTR kontroll,
 *    seadistus eraldi failis Sbr_Cfg.h.
 *  - ISO 26262: määratud ohutu olek, sisendite vahemiku ja vanuse kontroll,
 *    nõuete tunnused (REQ-SBR-xxx) jälgitavuseks testideni.
 */
#include "Sbr.h"       /* mooduli liides: tüübid ja sisendite koodid   */
#include "Sbr_Cfg.h"   /* seadistus: lävi, ajalimiit, kiiruse piirid   */

/* ------------------------------------------------------------------------
*/
/* Sisemiste funktsioonide prototüübid.
*/
/* static = nähtav ainult selles failis (MISRA reeglid 8.2 ja 8.8).
*/
/* ------------------------------------------------------------------------
*/
static uint8 Sbr_CheckSignals(const Sbr_InputType *input);
static Sbr_StateType Sbr_SelectState(const Sbr_InputType *input, uint8
faults);
static void Sbr_WriteOutput(Sbr_StateType state, uint8 faults,
                            Sbr_OutputType *output);

/* ------------------------------------------------------------------------
*/
/* Sisemised funktsioonid
*/
/* ------------------------------------------------------------------------
*/

/**
 * @brief  Kontrollib kõiki sisendeid: sõnumi vanust ja lubatud väärtusi.
 * @param[in] input  sisendsignaalid (ei tohi olla NULL_PTR)
 * @return Vigade bitimask. SBR_FAULT_NONE tähendab, et kõik on korras.
 *
 * REQ-SBR-003
```

```
 */
static uint8 Sbr_CheckSignals(const Sbr_InputType *input)
{
    /* Alguses vigu pole. Muutuja saab väärtuse kohe (MISRA reegel 9.1). */
    uint8 faults = SBR_FAULT_NONE;

    /* Sõnum vanem kui 200 ms -> andmeid ei saa usaldada.
     * Täpselt 200 ms on veel lubatud, sest reegel ütleb "vanem kui". */
    if (input->age_ms > SBR_MSG_TIMEOUT_MS)
    {
        faults |= SBR_FAULT_TIMEOUT;   /* lisab biti, teisi bitte ei muuda */
    }

    /* Süüde: lubatud on ainult 0 ja 1, kõik muu (2...255) on viga. */
    switch (input->ignition_on)
    {
        case SBR_IGN_OFF:
        case SBR_IGN_ON:
            break;                     /* kehtiv väärtus, viga pole */

        default:
            faults |= SBR_FAULT_IGNITION;
            break;
    }

    /* Turvavöö: lubatud on 0 ja 1. Anduri viga (2) ja tundmatu kood
     * on mõlemad vead. */
    switch (input->belt_state)
    {
        case SBR_BELT_UNBUCKLED:
        case SBR_BELT_BUCKLED:
            break;                     /* kehtiv väärtus, viga pole */

        case SBR_BELT_FAULT:
        default:
            faults |= SBR_FAULT_BELT;
            break;
    }

    /* Iste: lubatud on 0 ja 1. Anduri viga (2) ja tundmatu kood
     * on mõlemad vead. */
    switch (input->seat_state)
    {
        case SBR_SEAT_EMPTY:
        case SBR_SEAT_OCCUPIED:
            break;                     /* kehtiv väärtus, viga pole */
```

```
        case SBR_SEAT_FAULT:
        default:
            faults |= SBR_FAULT_SEAT;
            break;
    }

    /* Kiirus: viga, kui väärtus puudub (0xFFFF) või on üle 300 km/h,
     * mis on füüsikaliselt ebausutav. */
    if ((input->speed_raw == SBR_SPEED_RAW_NOT_AVAILABLE) ||
        (input->speed_raw > SBR_SPEED_RAW_MAX_VALID))
    {
        faults |= SBR_FAULT_SPEED;
    }

    return faults;   /* üks väljumispunkt (MISRA reegel 15.5) */
}

/**
 * @brief  Valib hoiatuse oleku sisendite ja vigade maski põhjal.
 * @param[in] input   sisendsignaalid (ei tohi olla NULL_PTR)
 * @param[in] faults  funktsiooni Sbr_CheckSignals() tulemus
 * @return Valitud olek.
 *
 * Järjekord: 1) vead -> ohutu olek, 2) hoiatuse tingimused, 3) hoiatust
pole.
 */
static Sbr_StateType Sbr_SelectState(const Sbr_InputType *input, uint8
faults)
{
    Sbr_StateType state;   /* saab väärtuse igas allolevas harus */

    if (faults != SBR_FAULT_NONE)
    {
        /* REQ-SBR-003: vigane või aegunud signaal -> tuli (ohutu olek).
         * Heli jääb välja, sest vigaste andmetega ei saa kinnitada,
         * et vöö on tegelikult lahti. */
        state = SBR_STATE_DEGRADED;
    }
    else if ((input->ignition_on == SBR_IGN_ON) &&
             (input->seat_state == SBR_SEAT_OCCUPIED) &&
             (input->belt_state == SBR_BELT_UNBUCKLED))
    {
        /* REQ-SBR-001: süüde sees JA iste hõivatud JA vöö lahti
         * -> hoiatus on vajalik. */
        if (input->speed_raw > SBR_SPEED_AUDIO_THRESHOLD_RAW)
```

```
        {
            /* REQ-SBR-002: üle 10 km/h -> tuli + heli. */
            state = SBR_STATE_VISUAL_AUDIO;
        }
        else
        {
            /* REQ-SBR-002: alati vähemalt tuli (ka täpselt 10,00 km/h). */
            state = SBR_STATE_VISUAL;
        }
    }
    else
    {
        /* REQ-SBR-001: süüde väljas, iste tühi või vöö kinni
         * -> hoiatust pole vaja. */
        state = SBR_STATE_NO_WARNING;
    }

    return state;   /* üks väljumispunkt (MISRA reegel 15.5) */
}

/**
 * @brief  Teisendab oleku väljundkäskudeks: tuli, heli, olek ja vead.
 * @param[in]  state   valitud olek
 * @param[in]  faults  vigade bitimask
 * @param[out] output  väljund (ei tohi olla NULL_PTR)
 *
 * Tundmatu oleku väärtus satub default-harusse, mis kirjutab ohutu oleku
 * (kaitsev programmeerimine, MISRA reegel 16.4).
 */
static void Sbr_WriteOutput(Sbr_StateType state, uint8 faults,
                            Sbr_OutputType *output)
{
    switch (state)
    {
        case SBR_STATE_NO_WARNING:
            /* Hoiatust pole: tuli ja heli väljas. */
            output->visual_on = FALSE;
            output->audio_on  = FALSE;
            output->state     = SBR_STATE_NO_WARNING;
            break;

        case SBR_STATE_VISUAL:
            /* Ainult tuli. */
            output->visual_on = TRUE;
            output->audio_on  = FALSE;
            output->state     = SBR_STATE_VISUAL;
```

```
            break;

        case SBR_STATE_VISUAL_AUDIO:
            /* Tuli + heli. */
            output->visual_on = TRUE;
            output->audio_on  = TRUE;
            output->state     = SBR_STATE_VISUAL_AUDIO;
            break;

        case SBR_STATE_DEGRADED:
        default:
            /* Ohutu olek: tuli SEES, heli VÄLJAS. Siia jõuab ka iga
             * tundmatu väärtus (nt kui muutuja on mälus rikutud). */
            output->visual_on = TRUE;
            output->audio_on  = FALSE;
            output->state     = SBR_STATE_DEGRADED;
            break;
    }

    /* Vigade mask läheb alati väljundisse, et diagnostika näeks põhjust. */
    output->faults = faults;
}

/* ----------------------------------------------------------------------
*/
/* Avalik funktsioon
*/
/* ----------------------------------------------------------------------
*/

/* Kirjeldus on failis Sbr.h. */
Std_ReturnType Sbr_Evaluate(const Sbr_InputType *input, Sbr_OutputType
*output)
{
    /* Algväärtused on ohutud: kui midagi läheb valesti, jääb ohutu olek. */
    Std_ReturnType ret    = E_NOT_OK;
    Sbr_StateType  state  = SBR_STATE_DEGRADED;    /* ohutu olek       */
    uint8          faults = SBR_FAULT_NULL_INPUT;  /* "sisend puudub"  */

    /* Sisendeid hinnatakse ainult siis, kui sisendi viit on olemas. */
    if (input != NULL_PTR)
    {
        faults = Sbr_CheckSignals(input);          /* 1. vigade tuvastus */
        state  = Sbr_SelectState(input, faults);   /* 2. hoiatuse valik  */
        ret    = E_OK;
    }
```

```
    /* Väljundisse kirjutatakse ainult siis, kui väljundi viit on olemas.
     * NULL_PTR-i kaudu kirjutamine kõrvaldaks programmi. */
    if (output != NULL_PTR)
    {
        Sbr_WriteOutput(state, faults, output);    /* 3. väljundi kirjutamine
*/
    }
    else
    {
        ret = E_NOT_OK;   /* kutsuja saab teada, et väljund puudus */
    }

    return ret;   /* E_OK ainult siis, kui mõlemad viidad olid korras */
}
```

test_Sbr.c

```
/**
 * @file    test_Sbr.c
 * @brief   Turvavöö meeldetuletuse (SBR) mooduli ühiktestid.
 *
 * Testkood ei ole tootekood, seega võib see kasutada <stdio.h>-d
 * (MISRA C:2012 reegel 21.6 kehtib mooduli, mitte testide kohta).
 *
 * Kolm osa:
 *  1. Nõuetepõhised testid tabelina (igal real nõude tunnus REQ-SBR-xxx),
 *     sh piirväärtused (ISO 26262-6, tabel 8).
 *  2. API testid: mis juhtub NULL_PTR argumentidega.
 *  3. Back-to-back test: iga võimalikku süüte/vöö/istme koodi ja iga
 *     kiiruse väärtust võrreldakse lihtsa referentsmudeliga.
 */
#include <stdio.h>      /* printf: tulemuste väljastamine */
#include "Sbr.h"        /* testitav moodul */
#include "Sbr_Cfg.h"    /* seadistus */

/* Ühe testjuhtumi kirjeldus: sisendid ja oodatav tulemus. */
typedef struct
{
    const char    *req;         /* nõude tunnus, nt "REQ-SBR-001"          */
    const char    *name;        /* testi lühinimi (väljastatakse ekraanile)
*/
    Sbr_InputType  in;          /* süüde, vöö, iste, speed_raw, age_ms
*/
    boolean        exp_visual;  /* oodatav tuli
*/
```

```
    boolean        exp_audio;   /* oodatav heli
*/
    Sbr_StateType  exp_state;   /* oodatav olek
*/
    uint8          exp_faults;  /* oodatav vigade mask
*/
} TestCase;

/* Lühikesed nimed, et testitabel mahuks ära ja oleks loetav. */
#define OFF   SBR_IGN_OFF          /* süüde väljas   */
#define ON    SBR_IGN_ON           /* süüde sees     */
#define UNB   SBR_BELT_UNBUCKLED   /* vöö lahti      */
#define BUCK  SBR_BELT_BUCKLED     /* vöö kinni      */
#define EMPTY SBR_SEAT_EMPTY       /* iste tühi      */
#define OCC   SBR_SEAT_OCCUPIED    /* iste hõivatud  */

#define S_NONE  SBR_STATE_NO_WARNING     /* hoiatust pole */
#define S_VIS   SBR_STATE_VISUAL         /* ainult tuli   */
#define S_AUD   SBR_STATE_VISUAL_AUDIO   /* tuli + heli   */
#define S_DEG   SBR_STATE_DEGRADED       /* ohutu olek    */

/* Testitabel. Iga rida: nõue, nimi, { süüde, vöö, iste, kiirus, vanus },
 * oodatav tuli, oodatav heli, oodatav olek, oodatav vigade mask. */
static const TestCase g_cases[] =
{
    /* --- REQ-SBR-001: hoiatus ainult siis, kui süüde sees + iste hõivatud +
vöö lahti.
     *     Kõik 8 kombinatsiooni, kiirus 0. */
    { "REQ-SBR-001", "ign OFF, empty, unbuckled",        { OFF, UNB,  EMPTY,
0U, 0U }, FALSE, FALSE, S_NONE, SBR_FAULT_NONE },
    { "REQ-SBR-001", "ign OFF, empty, buckled",          { OFF, BUCK, EMPTY,
0U, 0U }, FALSE, FALSE, S_NONE, SBR_FAULT_NONE },
    { "REQ-SBR-001", "ign OFF, occupied, unbuckled",     { OFF, UNB,  OCC,
0U, 0U }, FALSE, FALSE, S_NONE, SBR_FAULT_NONE },
    { "REQ-SBR-001", "ign OFF, occupied, buckled",       { OFF, BUCK, OCC,
0U, 0U }, FALSE, FALSE, S_NONE, SBR_FAULT_NONE },
    { "REQ-SBR-001", "ign ON, empty, unbuckled",         { ON,  UNB,  EMPTY,
0U, 0U }, FALSE, FALSE, S_NONE, SBR_FAULT_NONE },
    { "REQ-SBR-001", "ign ON, empty, buckled",           { ON,  BUCK, EMPTY,
0U, 0U }, FALSE, FALSE, S_NONE, SBR_FAULT_NONE },
    { "REQ-SBR-001", "ign ON, occupied, unbuckled",      { ON,  UNB,  OCC,
0U, 0U }, TRUE,  FALSE, S_VIS,  SBR_FAULT_NONE },
    { "REQ-SBR-001", "ign ON, occupied, buckled",        { ON,  BUCK, OCC,
0U, 0U }, FALSE, FALSE, S_NONE, SBR_FAULT_NONE },

    /* --- REQ-SBR-002: tuli alati, heli üle 10 km/h. Piirväärtused 9,99 /
```

```
10,00 / 10,01 km/h. */
    { "REQ-SBR-002", "9.99 km/h -> visual only",         { ON,  UNB,  OCC,
999U,   0U }, TRUE,  FALSE, S_VIS,  SBR_FAULT_NONE },
    { "REQ-SBR-002", "10.00 km/h -> visual only",        { ON,  UNB,  OCC,
1000U,  0U }, TRUE,  FALSE, S_VIS,  SBR_FAULT_NONE },
    { "REQ-SBR-002", "10.01 km/h -> visual + audio",     { ON,  UNB,  OCC,
1001U,  0U }, TRUE,  TRUE,  S_AUD,  SBR_FAULT_NONE },
    { "REQ-SBR-002", "300.00 km/h (max) -> audio",       { ON,  UNB,  OCC,
30000U, 0U }, TRUE,  TRUE,  S_AUD,  SBR_FAULT_NONE },
    { "REQ-SBR-002", "50 km/h, buckled -> none",         { ON,  BUCK, OCC,
5000U,  0U }, FALSE, FALSE, S_NONE, SBR_FAULT_NONE },
    { "REQ-SBR-002", "50 km/h, seat empty -> none",      { ON,  UNB,  EMPTY,
5000U,  0U }, FALSE, FALSE, S_NONE, SBR_FAULT_NONE },
    { "REQ-SBR-002", "50 km/h, ign OFF -> none",         { OFF, UNB,  OCC,
5000U,  0U }, FALSE, FALSE, S_NONE, SBR_FAULT_NONE },

    /* --- REQ-SBR-003: sõnumi vanus. Piirväärtused 200 / 201 ms. */
    { "REQ-SBR-003", "age 200 ms -> still valid",        { ON,  UNB,  OCC,
5000U, 200U }, TRUE, TRUE,  S_AUD, SBR_FAULT_NONE },
    { "REQ-SBR-003", "age 201 ms -> safe state",         { ON,  UNB,  OCC,
5000U, 201U }, TRUE, FALSE, S_DEG, SBR_FAULT_TIMEOUT },
    { "REQ-SBR-003", "age max -> safe state",            { ON,  UNB,  OCC,
0U, 0xFFFFFFFFU }, TRUE, FALSE, S_DEG, SBR_FAULT_TIMEOUT },
    { "REQ-SBR-003", "stale + buckled -> safe state",    { ON,  BUCK, OCC,
0U, 201U }, TRUE, FALSE, S_DEG, SBR_FAULT_TIMEOUT },

    /* --- REQ-SBR-003: vigased signaalid. Iga signaal eraldi + mitu viga
korraga. */
    { "REQ-SBR-003", "ignition = 2 (invalid)",           { 2U,    BUCK, OCC,
0U, 0U }, TRUE, FALSE, S_DEG, SBR_FAULT_IGNITION },
    { "REQ-SBR-003", "ignition = 0xFF (n/a)",            { 0xFFU, BUCK, OCC,
0U, 0U }, TRUE, FALSE, S_DEG, SBR_FAULT_IGNITION },
    { "REQ-SBR-003", "belt sensor FAULT",                { ON,
SBR_BELT_FAULT, OCC, 0U, 0U }, TRUE, FALSE, S_DEG, SBR_FAULT_BELT },
    { "REQ-SBR-003", "belt = 0xFF (n/a)",                { ON, 0xFFU, OCC,
0U, 0U }, TRUE, FALSE, S_DEG, SBR_FAULT_BELT },
    { "REQ-SBR-003", "seat sensor FAULT",                { ON, BUCK,
SBR_SEAT_FAULT, 0U, 0U }, TRUE, FALSE, S_DEG, SBR_FAULT_SEAT },
    { "REQ-SBR-003", "seat = 0xFF (n/a)",                { ON, BUCK, 0xFFU,
0U, 0U }, TRUE, FALSE, S_DEG, SBR_FAULT_SEAT },
    { "REQ-SBR-003", "speed 300.01 km/h (implausible)",  { ON, UNB,  OCC,
30001U,  0U }, TRUE, FALSE, S_DEG, SBR_FAULT_SPEED },
    { "REQ-SBR-003", "speed 0xFFFF (n/a)",               { ON, UNB,  OCC,
0xFFFFU, 0U }, TRUE, FALSE, S_DEG, SBR_FAULT_SPEED },
    { "REQ-SBR-003", "belt fault while ign OFF",         { OFF,
SBR_BELT_FAULT, EMPTY, 0U, 0U }, TRUE, FALSE, S_DEG, SBR_FAULT_BELT },
```

```
    { "REQ-SBR-003", "several faults at once",           { ON,
SBR_BELT_FAULT, OCC, 0xFFFFU, 500U }, TRUE, FALSE, S_DEG,
      (uint8)(SBR_FAULT_BELT | SBR_FAULT_SPEED | SBR_FAULT_TIMEOUT) },
};

static unsigned int g_run    = 0U;   /* tehtud kontrollide arv     */
static unsigned int g_failed = 0U;   /* ebaõnnestunud kontrollide arv */

/* Tagastab oleku nime tekstina (ekraanile väljastamiseks). */
static const char *StateName(Sbr_StateType s)
{
    const char *name;
    switch (s)
    {
        case SBR_STATE_NO_WARNING:   name = "NO_WARNING";   break;
        case SBR_STATE_VISUAL:       name = "VISUAL";       break;
        case SBR_STATE_VISUAL_AUDIO: name = "VISUAL_AUDIO"; break;
        case SBR_STATE_DEGRADED:     name = "DEGRADED";     break;
        default:                     name = "?";            break;
    }
    return name;
}

/* Loendab kontrolli ja väljastab teate, kui kontroll ebaõnnestus. */
static void Check(boolean ok, const char *req, const char *name)
{
    g_run++;
    if (!ok)
    {
        g_failed++;
        printf("  FAIL  [%s] %s\n", req, name);
    }
}

/* ------------------------------------------------------------------ */
/* 1. Nõuetepõhised testid                                            */
/* ------------------------------------------------------------------ */
static void RunTableTests(void)
{
    unsigned int i;
    /* Testjuhtumite arv = tabeli suurus / ühe rea suurus. */
    const unsigned int n = (unsigned int)(sizeof(g_cases) /
sizeof(g_cases[0]));

    printf("1) Requirement-based tests\n");
    printf("   %-12s %-34s %-6s %-6s %-13s %s\n",
```

```
           "Req", "Case", "Visual", "Audio", "State", "Result");

    /* Käime tabeli läbi rida haaval. */
    for (i = 0U; i < n; i++)
    {
        const TestCase *tc = &g_cases[i];
        Sbr_OutputType  out;
        /* Kutsume testitavat funktsiooni selle rea sisenditega. */
        Std_ReturnType  ret = Sbr_Evaluate(&tc->in, &out);
        /* Test läbib ainult siis, kui KÕIK väljundid on oodatud. */
        boolean ok = (ret == E_OK) &&
                     (out.visual_on == tc->exp_visual) &&
                     (out.audio_on  == tc->exp_audio)  &&
                     (out.state     == tc->exp_state)  &&
                     (out.faults    == tc->exp_faults);

        /* Väljastame ühe tulemusrea. */
        printf("   %-12s %-34s %-6s %-6s %-13s %s\n",
               tc->req, tc->name,
               out.visual_on ? "ON" : "off",
               out.audio_on  ? "ON" : "off",
               StateName(out.state),
               ok ? "PASS" : "FAIL");
        Check(ok, tc->req, tc->name);
    }
}

/* ------------------------------------------------------------------------ */
/* 2. API testid: NULL_PTR argumendid                                       */
/* ------------------------------------------------------------------------ */
static void RunApiTests(void)
{
    const Sbr_InputType valid = { ON, UNB, OCC, 0U, 0U };   /* korrektne
sisend */
    Sbr_OutputType out;
    Std_ReturnType ret;

    printf("\n2) API robustness tests\n");

    /* Täidame väljundi meelega valede väärtustega, et näha, kas funktsioon
     * kirjutab sinna ohutu oleku. */
    out.visual_on = FALSE;
    out.audio_on  = TRUE;
    out.state     = SBR_STATE_NO_WARNING;
    out.faults    = SBR_FAULT_NONE;
```

```
    /* Sisend puudub -> oodatakse E_NOT_OK ja ohutut olekut. */
    ret = Sbr_Evaluate(NULL_PTR, &out);
    Check((ret == E_NOT_OK) && (out.visual_on == TRUE) && (out.audio_on ==
FALSE) &&
          (out.state == SBR_STATE_DEGRADED) && (out.faults ==
SBR_FAULT_NULL_INPUT),
          "API", "NULL input -> E_NOT_OK + safe state");

    /* Väljund puudub -> oodatakse E_NOT_OK, programm ei tohi kokku joosta.
*/
    ret = Sbr_Evaluate(&valid, NULL_PTR);
    Check(ret == E_NOT_OK, "API", "NULL output -> E_NOT_OK, no crash");

    /* Mõlemad puuduvad -> E_NOT_OK. */
    ret = Sbr_Evaluate(NULL_PTR, NULL_PTR);
    Check(ret == E_NOT_OK, "API", "both NULL -> E_NOT_OK, no crash");

    /* Mõlemad olemas -> E_OK. */
    ret = Sbr_Evaluate(&valid, &out);
    Check(ret == E_OK, "API", "valid arguments -> E_OK");

    printf("   4 checks done\n");
}

/* ------------------------------------------------------------------------ */
/* 3. Back-to-back test: võrdlus sõltumatu referentsmudeliga                */
/* ------------------------------------------------------------------------ */

/* Referentsmudel, kirjutatud otse ülesande tekstist ja võimalikult lihtsalt.
 * Kui moodul ja referents annavad sama tulemuse, on loogika õige. */
static void Reference(const Sbr_InputType *in, boolean *visual, boolean
*audio)
{
    boolean ign_ok   = (in->ignition_on <= 1U);       /* süüde 0 või 1
*/
    boolean belt_ok  = (in->belt_state  <= 1U);       /* vöö 0 või 1
*/
    boolean seat_ok  = (in->seat_state  <= 1U);       /* iste 0 või 1
*/
    boolean speed_ok = (in->speed_raw   <= 30000U);   /* kuni 300 km/h
*/
    boolean fresh    = (in->age_ms      <= 200U);     /* kuni 200 ms vana
*/

    if (!(ign_ok && belt_ok && seat_ok && speed_ok && fresh))
    {
```

```
        *visual = TRUE;            /* reegel 3: viga -> tuli */
        *audio  = FALSE;
    }
    else if ((in->ignition_on == 1U) && (in->seat_state == 1U) && (in-
>belt_state == 0U))
    {
        *visual = TRUE;            /* reeglid 1 + 2: tuli, heli üle 10 km/h
*/
        *audio  = (in->speed_raw > 1000U);
    }
    else
    {
        *visual = FALSE;           /* hoiatust pole */
        *audio  = FALSE;
    }
}

static unsigned long g_b2b_run  = 0UL;   /* võrreldud kombinatsioonide arv */
static unsigned long g_b2b_fail = 0UL;   /* erinevuste arv                 */

/* Võrdleb ühe sisendikombinatsiooni korral moodulit ja referentsi. */
static void CompareOne(const Sbr_InputType *in)
{
    Sbr_OutputType out;
    boolean ref_visual;
    boolean ref_audio;

    (void)Sbr_Evaluate(in, &out);              /* mooduli tulemus     */
    Reference(in, &ref_visual, &ref_audio);    /* referentsi tulemus  */
    g_b2b_run++;

    /* Tulemused peavad ühtima. Lisaks: heli ei tohi kunagi olla ilma tuleta.
*/
    if ((out.visual_on != ref_visual) || (out.audio_on != ref_audio) ||
        (out.audio_on && !out.visual_on))
    {
        /* Väljastame ainult esimesed 5 erinevust, et ekraan ei täituks. */
        if (g_b2b_fail < 5UL)
        {
            printf("  FAIL  B2B ign=%u belt=%u seat=%u speed=%u age=%lu\n",
                   (unsigned)in->ignition_on, (unsigned)in->belt_state,
                   (unsigned)in->seat_state, (unsigned)in->speed_raw,
                   (unsigned long)in->age_ms);
        }
        g_b2b_fail++;
    }
```

```
}

static void RunBackToBack(void)
{
    /* Kiiruse ja vanuse väärtused, mis on piiride lähedal. */
    static const uint16 speeds[] = { 0U, 1000U, 1001U, 30000U, 30001U,
0xFFFFU };
    static const uint32 ages[]   = { 0U, 200U, 201U, 0xFFFFFFFFU };
    Sbr_InputType in;
    unsigned int ign;
    unsigned int belt;
    unsigned int seat;
    unsigned int s;
    unsigned int a;
    unsigned long speed;

    printf("\n3) Back-to-back test vs. reference model\n");

    /* a) Iga süüte/vöö/istme kood (igaüks 0...255), valitud kiirused ja
vanused.
     *    256 x 256 x 256 x 10 = ~168 miljonit kombinatsiooni. */
    for (ign = 0U; ign <= 255U; ign++)
    {
        for (belt = 0U; belt <= 255U; belt++)
        {
            for (seat = 0U; seat <= 255U; seat++)
            {
                in.ignition_on = (uint8)ign;
                in.belt_state  = (uint8)belt;
                in.seat_state  = (uint8)seat;
                /* Kõik valitud kiirused, sõnum värske. */
                for (s = 0U; s < (sizeof(speeds) / sizeof(speeds[0])); s++)
                {
                    in.speed_raw = speeds[s];
                    in.age_ms    = 0U;
                    CompareOne(&in);
                }
                /* Kõik valitud vanused, kiirus 50 km/h. */
                in.speed_raw = 5000U;
                for (a = 0U; a < (sizeof(ages) / sizeof(ages[0])); a++)
                {
                    in.age_ms = ages[a];
                    CompareOne(&in);
                }
            }
        }
```

```
    }

    /* b) Iga kiiruse väärtus 0...65535 iga kehtiva süüte/vöö/istme
     *    kombinatsiooni ja iga valitud vanuse korral. */
    for (speed = 0UL; speed <= 0xFFFFUL; speed++)
    {
        for (ign = 0U; ign <= 1U; ign++)
        {
            for (belt = 0U; belt <= 1U; belt++)
            {
                for (seat = 0U; seat <= 1U; seat++)
                {
                    for (a = 0U; a < (sizeof(ages) / sizeof(ages[0])); a++)
                    {
                        in.ignition_on = (uint8)ign;
                        in.belt_state  = (uint8)belt;
                        in.seat_state  = (uint8)seat;
                        in.speed_raw   = (uint16)speed;
                        in.age_ms      = ages[a];
                        CompareOne(&in);
                    }
                }
            }
        }
    }

    printf("   %lu input combinations compared, %lu mismatches\n",
           g_b2b_run, g_b2b_fail);
    Check(g_b2b_fail == 0UL, "B2B", "implementation == reference model");
}

/* Testprogrammi algus: käivitab kõik kolm testiosa ja väljastab kokkuvõtte.
*/
int main(void)
{
    RunTableTests();
    RunApiTests();
    RunBackToBack();

    printf("\nSummary: %u checks, %u failed -> %s\n",
           g_run, g_failed, (g_failed == 0U) ? "ALL PASSED" : "FAILED");

    /* 0 = kõik testid läbisid (Makefile ja CI tunnevad selle ära). */
    return (g_failed == 0U) ? 0 : 1;
}
```

Makefile

```
# Kompilaator (vaikimisi gcc; clang-iga: make CC=clang test)
CC      ?= gcc

# Ranged kompilaatori hoiatused; -Werror muudab iga hoiatuse veaks.
CFLAGS  := -std=c99 -O2 -Wall -Wextra -Wpedantic -Wconversion -Wsign-
conversion \
           -Wshadow -Wcast-qual -Wstrict-prototypes -Wmissing-prototypes \
           -Wswitch-default -Wundef -Werror

# Need sihtmärgid ei ole failinimed.
.PHONY: all test coverage misra clean

# "make" ilma argumendita kompileerib testprogrammi.
all: test_Sbr

# Testprogramm kompileeritakse uuesti, kui mõni neist failidest muutub.
test_Sbr: Sbr.c test_Sbr.c Sbr.h Sbr_Cfg.h Std_Types.h
        $(CC) $(CFLAGS) -o $@ Sbr.c test_Sbr.c

# Kompileerib ja käivitab testid.
test: test_Sbr
        ./test_Sbr

# Mõõdab, kui suur osa Sbr.c ridadest ja harudest testidega läbiti (gcov).
coverage:
        $(CC) -std=c99 -O0 --coverage -o test_Sbr_cov Sbr.c test_Sbr.c
        ./test_Sbr_cov > /dev/null
        gcov -b -c -o . test_Sbr_cov-Sbr.gcno | head -5

# MISRA C:2012 kontroll cppchecki MISRA lisamooduliga (ainult moodul, mitte
testid).
misra:
        cppcheck --addon=misra --enable=style,warning,performance,portability
\
                 --std=c99 --error-exitcode=1 -q Sbr.c && echo "MISRA: no
findings"

# Kustutab kompileeritud ja katvuse failid.
clean:
        rm -f test_Sbr test_Sbr_cov *.gcda *.gcno *.gcov
```

Makefile'is peavad käsud algama tabulaatoriga, mitte tühikutega. Kui kopeerid koodi PDF-ist, kontrolli seda.